

WalletConnect Network Whitepaper*

WalletConnect Foundation

Abstract

The WalletConnect Network is the onchain UX ecosystem that makes web3 work by facilitating the use of any wallet across any app and platform. The Network also installed many of web3's UX standards which together foster today's composable and interoperable web3 ecosystem. WalletConnect today drives much of crypto's volume for millions of users across thousands of apps and wallets.

The WalletConnect Network is chain agnostic working across ecosystems from EVM and its L2s, to Solana, Cosmos, Polkadot, Bitcoin and more. To date it has facilitated 150 million connections for 24 million end users between over 600 different wallets and 40,000 applications. The next chapter for WalletConnect is the decentralization of the Network providing for continued resilience, privacy and censorship-resistance which will be governed by the WalletConnect Token (WCT). Beyond being a utility token to enable moving WalletConnect Network towards a permissionless state, the WCT token aligns web3's incentives towards, finally, prioritizing UX.

1 Introduction

The inception of WalletConnect was to solve a User Experience (UX)/Developer Experience (DX) problem: dapps were written for desktop but many end-users wanted to use wallets on their mobile phones. Developers wanted a uniform way to support all users' wallets. The WalletConnect relay was born together with the QR code that millions of users in web3 have successfully used to connect their wallets into the applications they want to use, restoring choice and composability for the industry.

Further UX/DX problems have shaped WalletConnect since then: enabling users to use their wallets in mobile apps/mobile browsers, enabling users to use

*IMPORTANT NOTE: The information provided in this whitepaper represents currently envisioned plans and goals. The strategies, details and goals in this paper are subject to change due to rapidly evolving technical, legal, regulatory and industry developments. Plans and goals may need to be quickly changed to adapt to new opportunities and challenges. Reown inc. and WalletConnect Foundation are contributors to the WalletConnect Network. Whether they and the Network are successful in achieving their goals may depend on factors outside of their control, including the actions of other contributors to and participants of the Network.

multiple extension wallets, creating a simpler one-click Sign-in With Ethereum (SIWE) experience while expanding all this to all networks beyond EVM, and many more.

Many facets of today's onchain UX/DX have been paved by WalletConnect, but a plethora of UX/DX issues remain in the status quo - from the need for an end user to know what "gas" is, holding assets on multiple chains, to an abundance of security issues and painful KYC processes. The WalletConnect Network's goal is to solve these issues to make web3 ready for the masses. All while honoring the industry's decentralization ethos.

The focus on these issues has made WalletConnect a part of many onchain experiences. WalletConnect serves millions of end-users and facilitates millions of dollars of onchain volume. WalletConnect is used everywhere and by everyone on any device: from a retail user in a developing country on a feature phone to a professional institutional trader in a bank on a desktop.

The core contributors of WalletConnect believe that without decentralization, web3's paradigms of self-custody, ownership, application composability, and end-user choice will be constrained - which will hurt adoption in the mid-to-long run. The next phase for WalletConnect is to both decentralize its infrastructure as well as to tackle the next set of crypto's UX issues that prevent the first billion users from using web3.

In preparation for this next chapter, what started as a single entity to develop both the Network and products, WalletConnect Inc. will be rebranded to Reown. Reown will continue to build AppKit and WalletKit - built on top of the WalletConnect Network - that enable builders to create onchain user experiences that make digital ownership effortless, intuitive, and secure.

Earlier this year, the WalletConnect Foundation was created as an organization to support the growth, security, and decentralization of the WalletConnect Network and the expansion of the WalletConnect ecosystem.

2 History

WalletConnect was founded in 2018 with the goal of developing a protocol that would enable seamless connections between wallets and applications across all platforms. Over the past six years, the WalletConnect protocol has established itself as a critical infrastructure in the web3 space, supporting millions of users and thousands of applications as well as facilitating much of web3's volume. The protocol's ability to provide secure, end-to-end encrypted connections has been fundamental in creating interoperability for every wallet, every app and every chain. Based on the protocol, a network of participants and contributors are creating the WalletConnect Network.

Key milestones include:

- The expansion to over 40,000 applications and 600+ wallets, facilitating more than 150 million connections as of 2024.

- The consistent growth in daily remote connections, evidencing widespread adoption.
- The transition to a permissioned decentralized database supported by third-party node operators.

Having cemented its position as critical infrastructure for the future of the internet, it is now time to transition to decentralized infrastructure that upholds the principles of permissionless access and digital ownership. This evolution positions the WalletConnect Network at the forefront of the new internet, driving innovation in onchain user experience and digital identity.

2.1 Role in the New Internet

As the internet enters an era of ownership, the WalletConnect Network plays a crucial role in shaping the landscape of the new, decentralized web. This evolution is characterized by several key factors:

- The rise of self-custody and digital ownership has led to a growing demand for products and services compatible with this new paradigm. Users are increasingly aware of the benefits of controlling their digital assets and data, driving a new generation of applications built on decentralized principles.
- Traditional industry leaders will recognize the importance of addressing this shift and should adapt their strategies to remain competitive.¹ They must understand that embracing decentralization and user ownership is not just a trend but a fundamental change in how the internet will operate.
- The composable nature of web3 has thus far only just begun to be seized by financial applications that aggregate/compose other apps to build better apps in a permissionless way. Composable consumer apps in social/travel/music/and all other industries have the potential to create a lot of new value.

However, mass adoption of these new technologies and principles can only be achieved by matching or surpassing previously set standards, particularly in terms of user experience and security. Today there is a chasm between web2 UX and web3 UX which presents a unique challenge and opportunity for the WalletConnect Network.

In this evolving ecosystem, applications are incentivized to create superior user experiences to capture and retain users in a vast ocean of options. Simultaneously, wallets must maintain the highest standards of security while ensuring they don't become a bottleneck for user experience.²

¹<https://consensys.io/blog/metamask-mastercard-and-baonex-unveil-revolutionary-way-to-pay-with-crypto-pr>

²<https://walletconnect.network/blog/walletguide-and-walletconnect-certified-the-future-of-digital-wallets>

The WalletConnect Network positions itself at the intersection of these needs, serving as a critical infrastructure layer between products and consumers. Its relay technology enables seamless connections between wallets (both desktop and mobile) and applications (web and mobile), providing users with choice and control over their digital interactions.

Furthermore, the Network’s decentralized database sets the stage for experiences previously not possible within the standards of the new internet. It allows for innovative applications and services that can leverage the benefits of decentralization while maintaining user-centric design principles.

Given the WalletConnect Network’s critical role in the web3 ecosystem, it is particularly susceptible to the risks associated with centralization. A centralized service, especially one offered for free, often comes with hidden costs - typically in the form of compromised privacy through data monetization or the risk of censorship through arbitrary offboarding. If bankrolled by an existing centralized entity, users might face privacy concerns due to potential data exploitation. Moreover, centralization creates a single point of failure, jeopardizing the Network’s resilience. The growing demand from the ecosystem, exemplified by wallets expressing interest in running nodes, further underscores the need and readiness for this transition.

The Network is designed to become more decentralized over time. This process involves a phased approach, gradually transitioning from a permissioned environment to a fully permissionless model. Throughout this evolution, the WalletConnect Network maintains its focus on user empowerment and security, helping it to remain a critical piece of infrastructure in the new internet landscape.

By facilitating secure, user-controlled interactions between applications and wallets, the WalletConnect Network is playing a pivotal role in enabling the transition to a more open, decentralized, and user-owned internet ecosystem.

3 WalletConnect Network Overview

The WalletConnect Network solves UX problems such as connecting users’ wallets to dapps in an end-to-end encrypted fashion. Beyond this, the Network promotes the use of standardized payloads to be used such as those defined by the Chain Agnostic Standards Alliance (CASA) which enables developers today to use the same interfaces no matter which network is used. As crypto reaches a more mainstream audience and serves Payments, KYC, and identity use-cases, this will be increasingly important as these are even less tied to specific networks than today’s use-cases.

Part of the beauty of the web is that its core data format, HTML, remains open. Microsoft and Adobe threatened this openness by forcing ActiveX and Flash onto users. The open web we have today was preserved through developers proactively building for the open web, and proactively downloading new browsers like Mozilla, despite Internet Explorer being pre-installed on Windows devices.

Web3’s HTML equivalent are CAIPs, chain-agnostic data formats specified by the Chain Agnostic Standards Alliance (CASA). All wallets using WalletConnect embrace these standards and they allow web3 to remain open. The emergence of proprietary alternatives come at the expense of privacy and censorship resistance - it is important for dapps and users to demand them to use open formats so that web3 remains open.

Since its inception, the vision has been that the WalletConnect Network should be decentralized. Not having a single entity control the Network improves its resilience and censorship resistance. A free centralized service results in “payment” in other ways - through loss of privacy because users’ data is being sold, or loss of censorship resistance because you can be offboarded. A decentralized network supported by the ecosystem is the only solution that truly meets the Web3 ethos.

The Network’s design emphasizes interoperability, connectivity, and composability, creating a user experience (UX) platform that is open and permissionless. This approach is essential for realizing the vision of the “new internet”—a decentralized and user-owned ecosystem.

4 Technology Overview

The WalletConnect Network’s services are more akin to classic web2 offchain infrastructure rather than blockchain: it provides webservices that require state and their operation is subject to low latency and high throughput requirements.

There was no off-the-shelf permissionless system available that was proven at scale and that met the latency requirements (see 4.2 below) which is why WalletConnect ended up marrying concepts of the blockchain and database worlds when building the Network.

4.1 Core Technology

The core technology for the WalletConnect Network is a permissionless rendezvous-hashing based database. Rendezvous hashing is a concept that all modern databases including Cassandra, DynamoDB, MongoDB, etc are based upon. It’s proven for global scale and it is decentralized by design as its lack of a central point of failure allows it to scale. However, none of these existing databases are permissionless as this is not a design goal they needed to achieve.

100% of WalletConnect’s production traffic has been served by the network since early 2024 with Service Nodes operated by Reown. In recent months Reown has brought additional entities into the permissioned federation of Service Node operations.

As such, WalletConnect Network also constitutes a new primitive in the decentralized application builders’ toolbox: a cost efficient/low latency/high throughput key value database serving WalletConnect which is one of web3’s most used applications.

One of the next frontiers of the system is full permissionless-ness. The Foundation, Reown and other collaborators expect to publish a technical paper explaining how the Network will move from its current permissioned state to an open and permissionless state.

4.2 Latency Considerations

Latency in Blockchains and similar permissionless systems is usually constrained as these systems, by design, broadcast all updates to all nodes. Despite innovation in the underlying, often Gossip-based broadcast algorithms and maturity of libraries such as libp2p, making it easier for developers to adopt them, these approaches still cannot overcome the physical constraints of achieving a global Quorum. Distributed Hashtable approaches such as Kademlia improve the average latency, but still suffer from poor worst-case latency. WalletConnect Network’s novel permissionless rendezvous hashing works through fulfilling nodes’ availability requirements through incentives and keeping reserve and standby nodes.

4.3 Relay Service

The Network’s Relay service, used to connect users’ wallets to dapps, is by design end-to-end encrypted. The Relay has no insight into users’ addresses, transaction hashes, KYC information, or any other information passed between the dapp and wallet. These features make the Network the perfect foundation for future use-cases such as payments where users can trust that middle-men cannot read what they are purchasing on the internet.

4.4 Phases of Decentralization

Initially, the Network operates in a permissioned environment where specific node operators manage the Service Nodes. The Gateway Nodes consume the database formed by the Service Nodes. It is expected that the Gateway Nodes will decentralize in phases similar to the Service Nodes, gradually allowing broader participation and ensuring Network resilience and security.

The Network is designed to operate in a fully permissionless manner, enabling any participant to run a Service Node or Gateway. It is expected that the Network will transition to a permissionless model following community consultation and technical validation to maintain Network integrity and performance.

5 Network Participants

The WalletConnect Network comprises various participants, each playing a critical role in maintaining the Network’s functionality and security. This section provides an overview of the key participants and their responsibilities within the Network. The Network protocol comprises Service Nodes and Gateway Nodes.

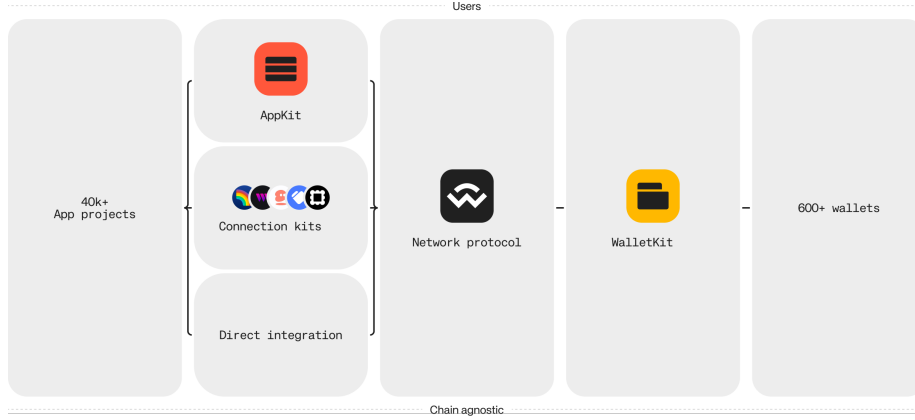


Figure 1: WalletConnect Network participants.

5.1 Service Node Operators

In the Service node architecture, clients are assumed to be always offline. For this reason, a “mailbox” persists messages such that they can fetch data when they come back online.

Service node operators manage the Service Nodes. The operators are tasked with maintaining the uptime and performance of these nodes, which is crucial for the Network’s scalability and robustness.

To participate, service node operators are required to stake WCT tokens. Operators earn rewards through a combination of staking rewards and performance-based incentives, which are calculated based on uptime and latency metrics.

5.1.1 Staking Requirements

To participate in the WalletConnect Network, node operators need to do two things:

1. Provide a pledged stake s in WCT tokens, which needs to be at least the minimum stake $S_{p\text{-MIN}}$ (see below)
2. Provide a timelock commitment T_L between 1 and 104 weeks

Both together define the actual stake-weight of the node. The stake-weight function is defined in Annex 10.4, which provides a detailed explanation of how the stake-weight is calculated based on the pledged stake and time lock commitment.

For the initial set of nodes (at TGE) the WalletConnect network requires a minimum staking amount $S_{p\text{-MIN}} = 100,000$ WCT and a minimum of 52 weeks of timelock.

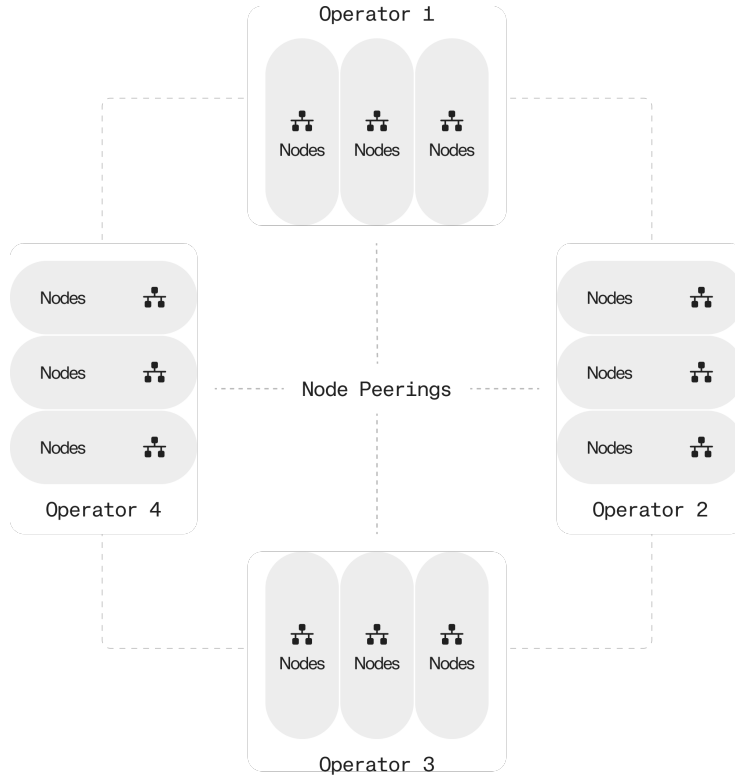


Figure 2: A Rendezvous Hash-ring in a federation of entities running service nodes.

5.1.2 Service Node Statuses

Nodes are categorized into different statuses—Active, Reserve, Standby, Jailed, and Deactivated—based on their performance and compliance with the Network’s requirements.

1. **Active:** These nodes participate in processing user requests and are part of a region. Initially this set entails at least 15, a number set by protocol governance impacting the overall node reward profitability. When a node engages in undesirable behavior (see service node slashing) a slashing event will be triggered against that node and it will be put into jailed status. Jailed status excludes a node from the active set of nodes.
2. **Jailed:** A node that triggers a slashing event will be excluded from the active set for a set time (24 hours) after which they move to the reserve status within which they are ranked based on their stake-weight.³

³This duration is equivalent to the length of a reward epoch. See Annex 10.1.

3. **Reserve:** These nodes operate as active nodes, i.e. meets the staking requirements (see 5.1.1) and a machine up and running which also participates in the replication process. When the size of the active set falls below 15, these nodes become active. Initially the target is 6 reserve nodes, such that active and reserve nodes make up 21 nodes.
4. **Standby:** A Node is in Standby status when it meets the staking requirements (see 5.1.1), but the Node Operator is not running a machine. They need to be ready, however, to run a machine when the size of the active and reserve set falls below 21 - if they fail to do so, when called to join the active set, they remain in the standby set.⁴
5. **Deactivate:** A node operator is free to decide to stop supporting the Network by triggering the deactivation event. Nodes in this deactivated set will not receive any more rewards, and need to wait until their timelock decreases to 0 to withdraw their stake.

This mechanism permits only reliable nodes to participate in processing user requests, maintaining the Network’s integrity and performance.

5.1.3 Service Node Performance

Node performance is a critical factor that directly impacts the overall efficiency, reliability, and scalability of the system. To evaluate individual node performance utilizing key metrics to generate a performance coefficient $U(i, t) \in [0, 1]$ for each node i at time t .

The evaluation framework incorporates two primary performance metrics:

1. **Uptime** (U_i): Defined as the percentage of time a node remains operational, calculated as the ratio of total operational time to the evaluation period.
2. **Latency** (L_i): Measured as the average response time for a node to process requests, computed by dividing the sum of response times by the number of messages.

To provide a nuanced assessment, a weighted scoring function is introduced that combines these metrics:

$$\text{Performance} = (W_u \cdot U_i) \cdot (W_l \cdot L_i)$$

Where W_u and W_l represent the weights assigned to uptime and latency, respectively. These weights can be adjusted based on Network priorities and requirements.

The WalletConnect Network employs a dynamic two-phase system to measure and verify node performance, facilitating the transition from a permissioned to a permissionless network.

⁴If in future developments delegation of stake is enabled those that have insufficient delegation would be included in the standby set.

1. **Permissioned Network (Phase 1):**

- Trusted nodes (oracles) for performance verification

2. **Transition to Permissionless Network (Phase 2):**

- All nodes participate in performance measurement

Key Transition:

- Phase 1: Trusted nodes (oracles) verify performance
- Phase 2: Every node pings and reports on every other node operator

In Phase 1, the oracle node plays a crucial role within the Network. It functions as both a data collector and a regular service operator. As a data collector, the oracle node gathers performance metrics from all active and reserve nodes in the Network. This phased approach allows for a controlled and secure evolution of the Network, balancing the need for initial stability with the long-term goal of decentralization.

5.1.4 Service Node Slashing

To maintain a high-performance network, the Network incorporates a slashing mechanism that penalizes underperforming nodes. This mechanism is directly tied to the previously defined node performance coefficient $U(i, t)$.

Slashing Threshold: The governance will define a performance threshold τ , where $0 < \tau < 1$. If a node's performance coefficient falls below this threshold, it triggers a slashing event.

The slashing process is as follows:

1. **Performance Evaluation:** The performance coefficient $U(i, t)$ is continuously calculated for each node i at regular intervals t .
2. **Threshold Check:** If $U(i, t) < \tau$, a slashing event is triggered for node i .
3. **Jailing:** Upon a slashing event, the underperforming node is immediately moved to the "jailed" set. This set contains nodes that are temporarily or permanently removed from active participation in the Network.
4. **Replacement:** Once a node is jailed, it is replaced in the active set by the next eligible node. This enables the Network to maintain its designated number of active nodes at all times.
5. **Stake Reduction:** The jailed node may face a reduction in its staked tokens as a penalty for underperformance. The exact percentage of stake reduction will be determined by Network governance.

6. **Reinstatement:** After serving the mandatory jailing period, the node will return to the standby set and subsequently to the reserve and active sets depending on the number of active nodes and performance of the rest of the Network.

By implementing this slashing mechanism, the Network can maintain a high standard of performance, ensuring that only the most reliable and efficient nodes participate in consensus and transaction processing.

5.2 Gateway Node Operators

Gateway node operators manage the Gateway Nodes, which are responsible for facilitating encrypted communications and data routing between wallets and applications. Initially, these nodes are operated by Reown to enhance stability and performance during the Network's early stages. Gateway nodes are responsible for:

- **Horizontal Scaling:** Gateway nodes are designed to handle increasing amounts of traffic by adding more nodes to the Network.
- **Velocity:** The Gateway Nodes optimize the speed of connections, reducing latency and improving the overall user experience when interacting with the WalletConnect Network.

It is expected that the WalletConnect Foundation will steward further decentralization of the operation of Gateway Nodes over time. The decentralization process involves moving project identifiers and metadata onto the Network, creating a clear distinction between wallets and applications, and introducing domain verification.

5.3 Wallets

Wallets enable users to manage their blockchain keys and interact with applications via the WalletConnect protocol. They play a vital role in the Network by enabling end-users to securely access and utilize blockchain services on any network. Wallets are responsible for integrating with the WalletConnect Network and providing a seamless user experience for managing digital assets and performing blockchain transactions.

Today, Reown provides the WalletKit sdk to enable simple integrations for wallets to the Network.

Wallets participating in the WalletConnect Network can earn rewards through staking and performance incentives. To qualify, wallets must stake WCT tokens, which enables them to participate in the Network's governance and earn staking rewards. The WalletConnect Certified program offers additional incentives for wallets that meet high standards of UX and integration, further encouraging wallets to stay up-to-date with the latest network features and best practices.

5.4 Apps

Apps within the web3 space are the products and services that drive traffic to the Network via direct integration or through any of the available SDKs. They play a crucial role in the WalletConnect Network by providing various services that end-users consume.

5.5 SDKs

SDKs act as proxies for applications, simplifying the connection process to the WalletConnect Network. They play a vital role in facilitating secure and efficient interactions between applications and the Network. Around 70% of connections into the WalletConnect Network come from SDKs. Due to their vital role in the Network, participating SDKs will receive a one-time allocation of WCT tokens. This allocation facilitates SDKs' participation in the Network through governance. SDKs are also incentivized to participate as node operators and fulfill multiple roles within the Network.

5.6 End Users

End users are the consumers of all the services within the WalletConnect Network, from wallets to applications, and through the relay and database nodes. They play a critical role in driving the Network's adoption and usage. End users can participate in the Network's governance through the WCT token.

Fostering a closer relationship between power users of web3 and app and wallet developers should enable tighter feedback loops such that UX can be iterated upon more quickly. The Network envisions that end users with WCT will be able to propose and vote on industry-wide UX improvements and product enhancements for the Network. A community-first innovation approach should facilitate that the evolution of onchain experiences meet real user needs and preferences. An example of this may be user-voted categories in WalletGuide. Knowing that WCT token holders are the most active Web3 users, it's also expected that apps in the Network can offer early access to these users in exchange for testing and feedback.

End users will enjoy a free service by the Network, to remove all barriers of entry and foster crypto adoption.

5.7 Technical Council

The Technical Council, elected by the Foundation and the community through governance, is envisioned to consist of core developers and node operators responsible for overseeing the technology and infrastructure of the WalletConnect Network. Their primary role is intended to provide Network stability, security, and continuous evolution.

The Technical Council would be tasked with maintaining the Network's core infrastructure, implementing upgrades, and ensuring the Network adapts to new

technological advancements. They will coordinate with the Foundation to align on the Network’s technical roadmap and governance. This collaborative effort is intended to ensure that the Network remains secure, scalable, and capable of supporting new features and integrations.

The Technical Council is expected to make contributions to the Network during the transition phases of the Network’s governance, gradually taking over technical responsibilities and facilitating a smooth shift towards a fully decentralized governance model. This phased approach allows for careful planning and execution, preparing the Network’s long-term success and sustainability.

6 WCT Token

The WCT token powers the onchain UX ecosystem, acting as both a reward and governance mechanism within the ecosystem. This section outlines the functionality, fee structures, and rewards associated with the WCT token.

The WCT token has four primary functions within the WalletConnect Network:

- **Fees:** At first the Network will not charge fees, but token holders will have the ability to vote on fees being charged for Network services such as relay usage and other future services.⁵
- **Rewards:** WCT tokens are distributed as rewards to incentivize participation and contribute to Network security and efficiency.
- **Staking:** Participants can stake WCT tokens to earn rewards and participate in governance.
- **Governance:** WCT holders can vote on proposals and changes, giving the community control over the Network’s development through decentralized governance.⁶

WCT token holders will have the power to influence the Network’s direction through governance mechanisms. They will be able to vote on proposals and changes to the Network, allowing the community to have a say in its evolution. This decentralized governance model is designed to foster a community-driven ecosystem.

6.1 Token Transferability

At launch, the WCT token will be non-transferable for all holders. This means that token holders will not be able to transfer their tokens to other wallets or addresses. However, this restriction does not affect then-enabled utilities of the WCT token such as governance participation and staking capabilities, allowing

⁵Fees may be introduced at a later time after approval by WCT tokenholders according to governance proposals and voting mechanisms.

⁶Governance participation will be introduced at a later stage.

all users, regardless of their allocation conditions, to participate in governance and staking activities.

6.1.1 Rationale for Non-Transferability

1. **Network Functionality:** The purpose of the WCT token is to support and enable network functionality. By restricting transferability, the tokens will remain within the ecosystem to serve their intended purpose.
2. **Long-Term Focus:** This purpose for the WCT token and restriction on transferability allows the prioritization of the long-term development and stability of the WalletConnect Network. Non-transferability provides an opportunity to thoroughly test all token functionalities, including staking, governance and mechanisms.
3. **Ecosystem Development:** This short-term non-transferability period serves as the starting point for creating and nurturing the WalletConnect ecosystem, allowing participants to engage with the Network's core functionalities.
4. **Infrastructure Protection:** As a key piece of web3 infrastructure, the WalletConnect Network must be stable and secure. This non-transferability period mitigates potential risks associated with premature token liquidity.

It is expected that the WalletConnect Foundation will facilitate the token holder community to propose and vote on when transferability will be enabled.

By taking this measured, community-involved approach, the WalletConnect Foundation aims to build a robust, secure, and sustainable ecosystem that serves the long-term interests of all participants in the WalletConnect Network.

6.2 WCT Allocation

The initial supply of WCT tokens is capped at 1 billion, with allocations designed to support the Network's growth and sustainability. These allocations include:

- **Core Development:** 7% For further development of the protocol and related modules
- **Rewards:** 17.5% For staking and performance rewards
- **Airdrops:** 18.5% For seasonal airdrops to users, apps, wallets, nodes, etc.
- **Team:** 18.5% For team members of Reown and WalletConnect.
- **Previous Backers:** 11.5% For those who provided resources and support to the early-stage Network.
- **WalletConnect Foundation:** 27% For partnerships, grants, ecosystem development, and operations.



Figure 3: WCT Allocations

Tokens allocated to core development, team and previous backers will be subject to a 4-year unlock including a 1 year cliff starting at the token generation event (TGE).

The Airdrops allocations will be released in seasonal airdrops starting at the token’s public launch and continuing throughout the following years.

6.3 Token Inflation

The initial design of the WalletConnect Network’s tokenomics does not include token inflation. The current model focuses on utilizing existing token allocations and the potential to introduce fee structures to support Network operations and incentivize participation such that inflation is not envisioned within the first 3-4 years.

However, the Network’s governance structure and community retain the flexibility to implement inflation mechanisms in the future if deemed appropriate. Any decision to introduce inflation would be subject to careful consideration of Network metrics, participant feedback, and overall ecosystem health, with specific parameters to be determined through Network governance processes.

6.4 Token Flow

As demonstrated by the consistent growth in remote connections, there is an increasing demand for decentralized products and services. This demand translates into computational bandwidth and storage capacity requirements, generating costs for each connection established through the Network.

The simplified token flow within the WalletConnect Network follows these steps:

1. A user initiates a connection with an app.
2. The user connects with the relay to establish the connection.
3. Once the connection is established, the relay facilitates communication.
4. Each message relayed has an associated price, which the app or SDK pays in fees.
5. The collected fees are used to reward nodes and wallets for their work in maintaining the Network.
6. This process repeats for all messages conveyed through the Network.

For an overview of the Network and token flow see Annex 10.3. Note that the mechanism for charging fees is intended to ensure that the end user does not have to pay to make a connection and has no other friction added to their user experience.

6.5 Fees & Fee Mechanisms

Initially, applications and SDKs will not be charged fees by the Network for using the relay services. However, given there is a cost to provide this service in a privacy-preserving and censorship resistant way (i.e. the costs aren't being covered by selling data or gatekeeping the ecosystem) it may be necessary, when the ecosystem reaches an appropriate stage of maturity, that the introduction of fees could be proposed by the community, discussed, and decided upon through governance mechanisms.

During this time, efforts are focused on encouraging adoption and growth, minimizing disruption to users, and allowing time for necessary infrastructure adjustments and progress toward future permissionless operation.

All the Network participants groups will be receiving WCT tokens (more on this below) to prepare them for when this is enabled. If additional functionality is layered into the Network, new fee models may emerge which will involve fees to be paid in WCT token.

6.6 Rewards

17.5% of the initial token supply is allocated for rewards to incentivize Network participants over the first few years of operations. This allocation is strategically phased: in the first year, only 5% will be distributed to test Network assumptions, while the remaining 12.5% is reserved for subsequent years. This larger portion is a “flexible incentive” and may be subject to change to maintain the reward mechanism’s support of Network goals and benefits to all participants.

6.6.1 Staking

Staking is an important component of the Network design to incentivise appropriate behaviors from Network participants such that they can be rewarded for long term alignment, and have stakes slashed as punishment for undesirable behaviors such as not fulfilling node uptime requirements or not adhering to user voted requirements on user experience.

Staking rewards in the Network are derived from the Reward pool. One can understand given a total rewards budget $R_G(t)$, a staking rewards budget $R_S(t)$ as the following:

$$R_s(t) = W_s R_G(t)$$

W_s is set by governance, with a proposed predefined range of 0.25-0.66.

Calculating the rewards Each staker i possesses a stake-weight $w(i, t)$ and a consequent share of the overall sum of stake-weights:

$$s_{stWeight}(i, t) = \frac{w(i, t)}{\sum w(j, t)}$$

We'll denote the total sum of stake-weight as ΣW and the total sum of staked tokens as ΣS .

To counter the concentration of such a stake-weighted reward scheme, there is a cap c_{MAX} imposed on the share of stake-weights (proposal: $c_{MAX} = \frac{1}{10}$), hence the effective share of stake-weights is:

$$s_{stWeight}(i, t) = \min\left(\frac{w(i, t)}{\Sigma W}, c_{MAX}\right)$$

For each staker i , the individual rewards are their stake-weight share multiplied with the staking rewards, which are a part of the general rewards budget, that is:

$$R_S(t) = W_S * R_G(t)$$

As mentioned earlier, W_S is set by governance. The individual staking rewards are then:

$$r_S(i, t) = s_{stWeight}(i, t) * R_S(t)$$

Fixed Staking The Fixed staking system allows participants to lock up their tokens for predetermined periods. This approach provides flexibility while providing for network stability and long-term commitment from participants.

Key features of the fixed staking system include:

1. **Flexible Duration:** Users can choose staking periods ranging from 1 week to 2 years (104 weeks), the duration users can stake are measured in weekly increments.

2. **Lock-up Period:** During the chosen staking period, staked tokens are inaccessible and cannot be withdrawn, ensuring network stability.
3. **Reward Calculation:** Staking rewards are calculated based on the duration of the stake, with longer commitments typically yielding higher rewards.

Unstaking and Unbonding To maintain network stability and prevent sudden liquidity shocks, the unstaking process includes the following features:

1. **Disabled Early Unstaking:** Users cannot withdraw their staked tokens before the end of their chosen lock-up period.
2. **Unbonding Period:** Upon completion of the staking period, users must wait for a 7-day unbonding period before tokens are returned to their wallet. This allows for a gradual transition and protects the Network from sudden large-scale withdrawals.

Auto-locking To support network participants who require active staking positions for participation or reward collection, there is an auto-locking feature:

1. **Continuous Staking:** This option automatically renews the staking position upon expiration.
2. **Stake Weight Preservation:** Auto-locking maintains the user's time-lock, ensuring uninterrupted participation in network activities and reward collection.
3. **Flexible Opt-out:** Users can disable auto-locking at any time, reverting to the standard un-bonding process at the end of their current staking period.

Re-Staking The staking mechanism also includes options for re-staking:

1. **Manual Re-staking:** Users can manually re-stake their rewards at the contract level, increasing their stake in the Network.
2. **Future Developments:** While automatic re-staking is not currently implemented, it is recognized as a potential future enhancement to improve user experience and increase network participation.

6.6.2 Node Rewards

The node rewards budget is split into two phases. The first phase tackles the WCT token non-transferability period. The implementation of a fixed-base reward structure results in the following token-base revenues:

1. 100,000 WCT are distributed as initial allocation and can be staked. Those are locked for the entire period of non-transferability.

2. B_n is a boost given to nodes to reflect on their performance and activity that is not based on WCT price.

The calculations are as follows:

$$B_n = \alpha + (\text{weight} \cdot \text{performance score})$$

And a multiplier such that:

$$M_n = \frac{100,000}{T}$$

with T being the period on which the initial allocation is locked. The boost is based on the weight of the node relative to other nodes. So, initially, if there are 15 nodes, each getting 100,000 WCT, they would each get 6.67% weight. This stake weight evolves if some nodes do not restake their boost.

Individual node rewards are conditional on the performance factor $U(i, t) \in [0, 1]$. In line with the performance factors for other groups, there is a set of KPIs defining this. Initially, it will be based on uptime and latency.

6.6.3 Wallet Performance Rewards

The WalletConnect Network implements a performance-based rewards system for wallet providers. This system incentivizes high-quality user experiences and active participation in the Network.

Wallets are categorized into three tiers based on their performance and certification status:

1. Not Eligible
2. Eligible
3. Certified Wallet

Qualification Criteria: To qualify for rewards, a wallet must meet the following requirements:

1. Stake a minimum amount of tokens into the protocol⁷
2. Achieve at least 100,000 successful connections and signatures in any given monthly period

⁷The required stake is proportional to the minimum airdrop distributed for applications, ensuring alignment between wallet providers and the broader ecosystem.

Performance Metrics: The total wallet rewards are calculated based on:

1. Number of connections established
2. Number of signatures performed
3. User experience (UX) standards, with WalletConnect Certified⁸ as the benchmark

Reward Pools and Distribution:

1. Eligible Wallets:

- Share a pool of 1,500,000 WCT tokens over the first year⁹
- Must meet the minimum performance threshold of 100,000 successful connections

2. Certified Wallets:

- Share an enhanced pool of 4,000,000 WCT tokens over the first year¹⁰
- Reward distribution is non-linear, favoring higher-performing wallets

Initially, the reward calculation and distribution process will be executed off-chain, with results posted on-chain for transparency and verification. This approach allows for flexibility in refining the system while maintaining decentralized record-keeping. After the first year, the performance pool will transition to a flexible rewards budget, allowing for dynamic adjustments based on Network growth and changing priorities.

7 Governance

The governance of the WalletConnect Network is structured to facilitate decentralization, transparency, and community participation. This section outlines the roles and responsibilities of the WalletConnect Foundation and the community governance model that guides the Network.

7.1 Foundation

The WalletConnect Foundation is tasked with stewarding the Network by promoting its adoption, use, and growth. The Foundation's responsibilities include overseeing of grants to stakeholders, supporting app, sdk and wallet development teams, and managing partnerships.

⁸<https://walletconnect.network/blog/walletguide-and-walletconnect-certified-the-future-of-digital-wallets> - long term the decision criteria for the certified programme could be governed by the community

⁹Allocation used for the first year of rewards

¹⁰Allocation used for the first year of rewards

7.2 Councils

The Councils include several curated groups of individuals who are responsible for the different functions that are either part of the foundation, core development teams, node operator teams or that work independently. Envisioned Councils include:

- **Technical Council** - responsible for the technology & infrastructure
- **Partnerships Council** - responsible for the partnerships & growth

This structure is projected to be implemented during the constitution of the community governance, more about this in the next section.

7.3 Community Governance

The WalletConnect Network is designed for a fully decentralized governance model managed by community governance. Further decentralization is expected to be facilitated by approved proposals of WCT tokenholders participating in the Network governance.

Optimally this transition occurs through planned, multiple phases. An example of such planned multiple phases follows, though the actual transition will depend on input and approval from WCT tokenholders:

1. Phase 1 - TGE Preparation:

- The WalletConnect Foundation is established and begins operations.
- The Foundation and Reown collaborate on the Network's technical, community, partnerships, and administrative governance.

2. Phase 2 - Foundation Transition:¹¹

- The Foundation establishes different councils to progressively take over various governance functions.
- The Foundation expands its programs and responsibilities over community, partnerships, and administration considerations for the Network.
- Community Governance (See above) arises from WCT token stakers who participate in the Network and its governance.

3. Phase 3 - Partnerships Transition:

- The Partnerships Council, elected through community governance, assumes a more prominent role in community initiatives, including marketing, business development, grants programs, education, developer relations, and events.

¹¹Token holders can participate in governance starting in Phase 2 after TGE by staking their WCT tokens, proposing changes, and voting on key issues, thereby shaping the future of the WalletConnect Network.

4. Phase 4 - Technology Transition:

- The Technical Council, elected through community governance, assumes responsibility for technical governance as the Network becomes permissionless.
- The Foundation coordinates this transition.

5. Phase 5 - Administration Transition:

- The Foundation requires community governance approval to establish annual budgets, review and elect councils, and handle other administrative responsibilities through voting by community governance delegates.

Over 150 million connections have been provided by WalletConnect to the web3 ecosystem over the last 6 years. Decentralization of the Network marks an important milestone in ensuring the continued provision of a resilient, privacy-preserving, censorship resistant UX ecosystem to underpin the onchain internet. The WalletConnect Foundation welcomes open-source contributions, new ecosystem participants and feedback from the community.

8 Future Work and Roadmap

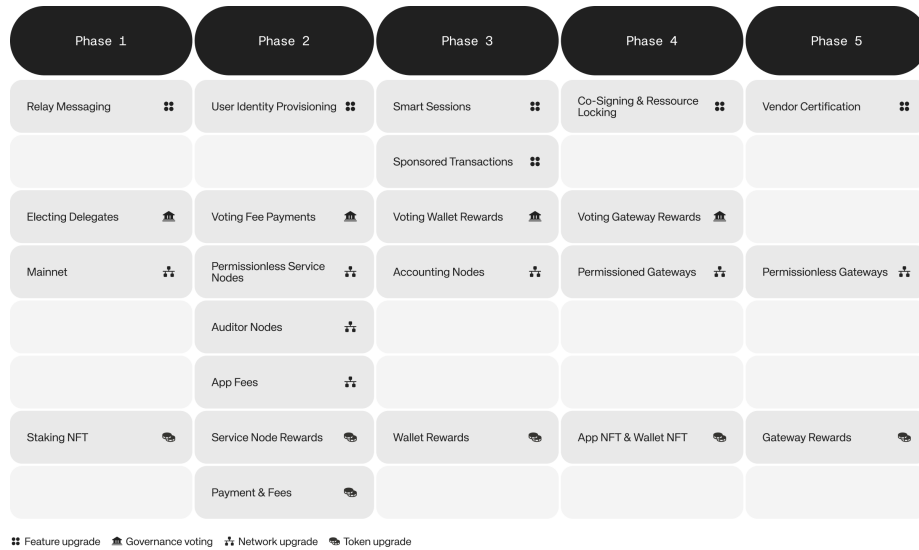


Figure 4: WalletConnect Network Roadmap

As WalletConnect continues to evolve, there are several key upgrades that will shape the future of the Network and the protocol. Stakeholders in the

WalletConnect Network can introduce upgrades. This roadmap outlines ideas proposed by core contributors to enhance decentralization, improve functionality, and introduce innovative services to meet the evolving needs of the web3 ecosystem.

Suggestions of core contributors for future upgrades include:

1. Permissionless Service Nodes

- Removing the permissioned process to run a service node
- Allowing anyone to become a service node operator, enhancing decentralization

2. Auditor Nodes

- Introduction of Auditor Nodes to monitor and report on service node performance and reliability

3. App Fee Payment

- Switching on fees where Apps/SDKs pay fees for Network usage
- Integration of fee payments with the general token dynamics, correlating rewards with the total fees paid by Apps

4. Accounting Nodes and Wallet Rewards

- Introduction of Accounting Nodes to independently distribute rewards to Wallets

5. Multiple Gateway Operators

- Introducing several Gateway servers to coordinate network activity will be a significant step towards further decentralization
- Coordination of App and wallet registrations through App NFT and Wallet NFT minting

8.1 Community-Driven Initiatives

As the WalletConnect Network continues to grow, community involvement could play an increasingly crucial role in shaping its future. The following examples are the types of initiatives that could be explored to foster community-driven innovation:

1. Seasonal Wallet Selection

- WCT token holders may have the opportunity to vote on user-choice wallets for WalletGuide each season, potentially ensuring that the most user-friendly and innovative wallets are highlighted.

2. Early Access Partnerships

- Exploration of collaborations with apps to potentially provide WCT token holders early access to new features.
- In return, apps might gain access to experienced UX testers and active web3 users, potentially creating a mutually beneficial ecosystem.

3. Beta Testing Program

- Consideration of a program where WCT holders could be given early access to beta-test cutting-edge features in wallets and apps.
- This program could potentially enable real-time feedback on usability, performance, and security, potentially driving rapid improvements.

4. Exclusive Feature Prototyping

- This program could potentially enable real-time feedback on usability, performance, and security, potentially driving rapid improvements.
- This initiative could potentially help shape the future of user experience by providing detailed input during prototype stages.

5. Crowdsourced UX Challenges

- This initiative could potentially help shape the future of user experience by providing detailed input during prototype stages.
- Top solutions could potentially be implemented, with contributors possibly recognized and rewarded with additional WCT tokens.

6. Exclusive Content and Alpha Access

- Top solutions could potentially be implemented, with contributors possibly recognized and rewarded with additional WCT tokens.
- Consideration of opportunities to participate in invite-only governance discussions or alpha testing sessions.

9 Glossary of Terms

Active Nodes Service nodes that are currently participating in processing user requests and are part of a region.

App Products and services within the web3 space that drive traffic to the WalletConnect Network via direct integration or through available SDKs.

Auditor Nodes Proposed future nodes to monitor and report on service node performance and reliability.

WCT Token The native token of the WalletConnect Network, used for fees, staking, and rewards.

Certified Wallet A category of wallets in the wallet performance rewards system that meet high standards of UX and integration.

Community Governance A form of governance for the WalletConnect Network where token holders can participate in decision-making.

Deactivated Nodes Service nodes that have stopped supporting the Network and no longer receive rewards.

End Users Consumers of all services within the WalletConnect Network, from wallets to applications.

Fixed Staking A staking system allowing users to lock their tokens for predetermined periods ranging from 1 week to 2 years.

Gateway Nodes Nodes forming web services that store state on the Network.

Jailed Nodes Service nodes that have triggered a slashing event and are excluded from the active set for a set time.

Performance Coefficient A measure of a node's performance based on up-time and latency metrics.

Relay Service The communication backbone of the WalletConnect Network, facilitating encrypted channels for various types of blockchain transactions and interactions.

Reserve Nodes Service nodes that operate as active nodes but are not currently in the active set.

SDK (Software Development Kit) Tools that simplify the connection process to the WalletConnect Network for applications.

Service Nodes Database nodes responsible for storing end-to-end encrypted messages and facilitating efficient data retrieval.

Slashing A penalty mechanism for underperforming nodes, potentially resulting in stake reduction and jailing.

Smart Session A feature that enables wallets to delegate signing power to Apps.

Sponsored Transaction Submission A proposed feature to standardize and simplify transaction submissions across wallets and services.

Standby Nodes Service nodes that meets the staking requirements (see 5.1.1) but are not currently running a machine.

Technical Council A group of core developers and node operators responsible for overseeing the technology and infrastructure of the WalletConnect Network.

Un-bonding Period A 7-day waiting period after the completion of a staking period before tokens are returned to a user's wallet.

WalletConnect Foundation The organization tasked with stewarding the WalletConnect Network, promoting its adoption, use, and growth.

WalletConnect Network A decentralized infrastructure designed to support the web3 ecosystem by facilitating seamless and secure interactions between applications and wallets.

Wallet Software that enables users to manage their blockchain keys and interact with applications via the WalletConnect protocol.

10 Annexes

10.1 Annex 1: Time Intervals

Time intervals ordered by length:

1. **Epoch_checkpoint:** 1h
 - The interval at which performance is checked
 - Potentially randomized, i.e., no fixed times for everyone
2. **Epoch_reward:** 1d
 - The interval over which rewards are assessed
 - The interval after which rewards are claimable

10.2 Annex 2: Request-based Pricing Formula

The formula assumes there is a minimal unit of work to measure network cost, called a request (comparable to opcode). Taking the minimum time-unit of service s (seconds), the capacity of the network C_{net} , and the cost of the network per such time-unit, $Cost_{net}$ the price of 1 request per second is:

$$P_{base} = Cost_{net}/C_{net}$$

To maintain a buffer, define $C_{max} < C_{net}$ as the maximum capacity for sale. The adjusted base price for break-even is:

$$P_{base} = Cost_{net}/C_{max}$$

For a given amount $r < C_{max}$ of requests per second, using a subsidy-factor $s \leq 1$:

$$P(r) = P_{base} \cdot (1 - s) \cdot r$$

This allows for fixed pricing (e.g., 0.0001\$ per request) while knowing the subsidy amount.

To make pricing more dynamic, four factors are considered:

1. Available capacity (f_a): Increases price as network utilization increases.
2. Timeslot (f_b): For when the rate-limit is supposed to start.
3. Timerange (f_c): Time until expiry.
4. Requested capacity (f_d): Decreases price per request for larger capacity requests (bulk discount).

Each factor is modeled as a value between 1 and $f_{a_{max}}$ (which can differ for each factor).

The aggregated pricing formula using these factors is:

$$P(f_a, f_b, f_c, f_d)(r) = P_{base} \cdot (1 - s) \cdot (f_a \cdot f_b \cdot f_c \cdot f_d)^{1/4} \cdot r$$

10.3 Annex 3: Token/Network Flow

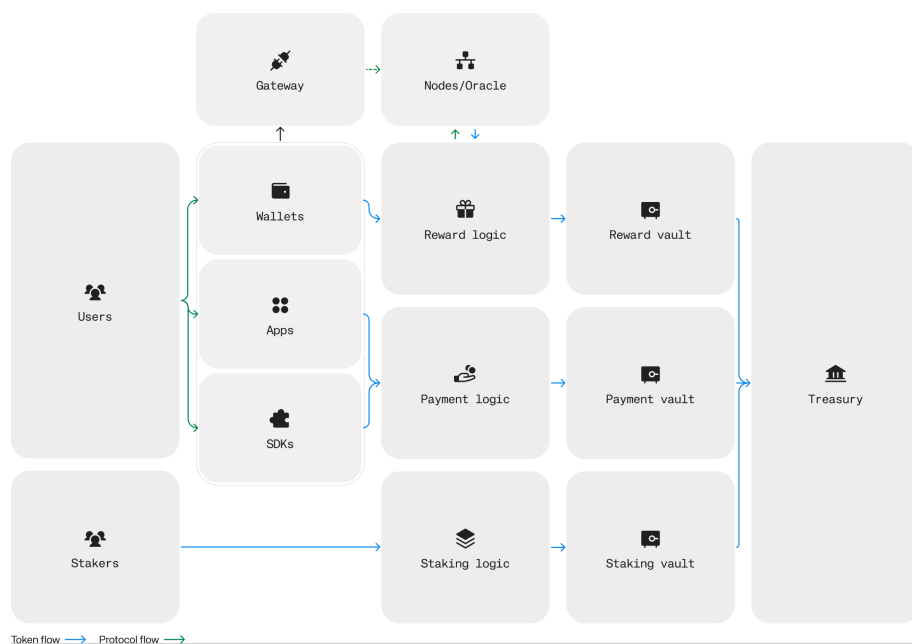


Figure 5: Overview of the Token/Network Flow

10.4 Annex 4: Stakeweight Formula

Each staker has to:

1. Provide a pledged stake s in WalletConnect tokens.
2. Provide a time lock commitment T_L between 1 and 208 weeks.

Both together define the actual stake-weight:

$$w(t) = f(s, T_L) = s \cdot \begin{cases} w_{\min} & \text{if } T_L < 1 \\ (1 - w_{\min}) \frac{T_L}{208} + w_{\min} & \text{if } 1 \leq T_L \leq 208 \\ 1 & \text{else} \end{cases} \quad (1)$$

Where:

- $w(i, t)$ is the stake-weight of staker i at time t
- s_i is the amount of tokens staked by staker i
- T_L is the time lock commitment in weeks

The stake-weight increases linearly with the time lock commitment, with a maximum bonus of 100% for a 4-year (208 weeks) commitment.

Mango Network

Transactional Omni-Chain Infrastructure Network

v1.2

Abstract

In response to the multiple pain points of Web3 application and DeFi protocol, such as fragmented user experience and fragmented liquidity, Mango Network proposes a transaction-based omni-chain infrastructure network solution, aiming to build a one-stop liquidity service network through main-chain contracts and modular contracts, and to bring users a more secure and trustworthy, asset-diversified, and convenient and autonomous transaction experience. At the same time, Mango, as Layer1 and supported by Move, provides developers and users with a secure, modular and high-performance Web3 infrastructure, and the Devnet boasts a high TPS of 297.45k, which is both standardized, scalable and interoperable, and provides a solid underlay for building non-custodial, decentralized, omni-chain mobility pledges, lending and borrowing, POS pledges, GameFi, and other core applications, so that participants can experience an unprecedented level of freedom in Web3.

I Web3.0 Public Chain Leader - Move

1.1 Move emergence background

In 2019, Meta (formerly Facebook) introduced the global circulation of the sovereign digital currency project Libra. Throughout this process, Libra and the Diem team left behind a valuable legacy - the Move language that addresses the shortcomings of Solidity and EVM, as well as the

derived public chain team. Move language is a resource-oriented programming language that borrows syntax and semantics from languages such as Rust, Cyclone, and Ada. The Move language aims to provide a high-performance, secure, and reliable programming environment for blockchain applications. It is a programming language designed specifically for digital assets.

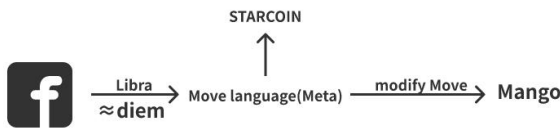
1.2 Advantages of Move series public chain

Move has redefined digital assets, believing that token assets are a unique and important form of data that should not be defined and represented using ordinary numerical types. Therefore, Move has created Resource to define on-chain assets separately.

Move language lowers the security threshold for developers, allowing contract developers to focus on business logic and write highly secure code, avoiding security vulnerabilities caused by low-level bugs. This is what makes DeFi (Decentralized Finance) safer and gives peace of mind to node users.

1.3 Characteristics of Move language

Move aims to provide a secure, efficient, and modular development solution for distributed applications. The core idea of the Move language is to break down complex programming tasks into multiple modules, effectively reducing complexity and lowering the probability of errors in programming.



1.3.1 Programming assets as first-class resources (First-class Resources)

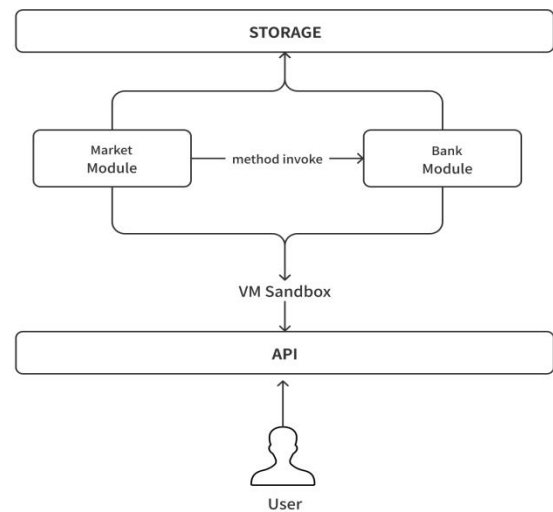
The Move language abstracts four attributes of resources: copyable, indexable, droppable, and storable. With different combinations of these attributes, users can easily define any type of resource.

The specified resources will be stored in a module controlled by the owner's account, represented by a verified owner/signer. The owner of these resources has the ultimate decision-making power and only the owner can determine the storage and transfer of the resources.

The design of resources allows for the transfer of digital assets to be a movement between storage locations rather than a simple addition or subtraction of balance values between accounts, thus avoiding re-entry and double-spending attacks.

1.3.2 Ensure its security through static call, virtual machine sandbox, etc

The Move language uses static calls, which improves runtime security, solves the problems of dynamic calls, and enhances network stability.



Move VM is the virtual machine sandbox for the Move programming language, where contract invocations are placed within the same sandbox. During this process, the security of the contract's state is primarily isolated through the internal security of the programming language, rather than relying on the virtual machine for isolation.

1.3.3 Verifiability

The Move programming language adopts formal verification. Formal verification is a means of using digital tools to analyze and prove the security of a program. The previously mentioned static calls and virtual machine sandbox can enhance the security of verification.

1.3.4 Flexibility

The flexibility of Move is reflected in the ability to freely combine various transactions through transaction scripts to achieve different functionalities, where one script can invoke multiple transactions. By using generic programming, Move ensures the scalability of contracts and increases code reusability.

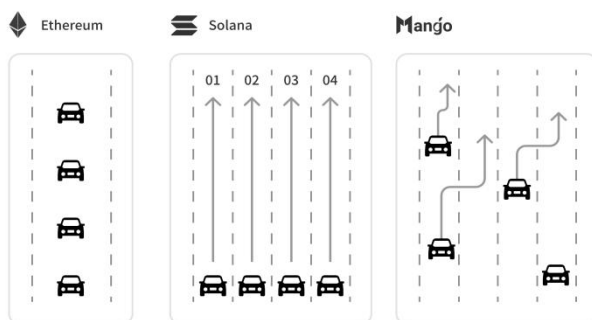
1.3.5 Contract composability

The composability of the Move language comes from the design of its Modules. Upgrading and optimizing Modules enables all other contracts that have used this Module to automatically use the latest version, accelerating the speed of upgrading and optimizing smart contracts using the Move language.

1.4 Status quo of Move series public chains

The Move language truly achieves the digitization of assets and is hailed as one of the most suitable languages for writing blockchain smart contracts. It has been compared to the current mainstream language Solidity multiple times and has surpassed it in many aspects. The main reason is that Move provides a more fitting handling of the core token assets in blockchain, compensating for Solidity's shortcomings in intuitiveness and security.

Comparison of Execution Engines



Currently, the public chain based on the Move language is in the stage of ecological expansion, attracting more developers and users to join. Once an ecosystem is formed, users can bring sustainable income. The more prosperous the ecosystem, the stronger the innovation ability, and the more marginal benefits there will be. This is also the reason why public chains can become the core

carrier of Web3.

II Technical Differences of Solidity VS Move

Transitioning from Bitcoin's Script to Ethereum's Solidity, undergoing the baptism of DeFi, Solidity has undoubtedly been successful. However, the security vulnerabilities of DeFi are also worth paying attention to. How to ensure the security of on-chain assets has become a major issue that the entire industry urgently needs to face.

Behind these vulnerabilities, VM is also constantly patching up, such as fixing the original overflow issues, but it is difficult to make substantial improvements in the underlying design flaws. Unlike previous development experiences, DeFi applications are essentially open-source and transparent financial systems that have extremely high security requirements. So, for financial scenarios, designing a more secure, reliable, and user-friendly smart contract language may be a better choice.

Characteristic	Solidity	Move
Turing completeness	Yes	Yes
Safety	Poor	Very good
Developer friendly	Yes	Yes
Dispersed storage	No	Yes
Engineering ability	Poor	Good
Flexibility	Dynamic mobilization	Generic programming
Financial scene enhancement	No	Yes

Solidity still has three major technical

challenges that need to be overcome urgently.

Solidity's syntax and expression methods are complex.

Traditional object-oriented programming languages like Java have become the best choice for novice developers to create Ethereum contracts. However, due to the complexity of Solidity, developers are unable to effectively understand its syntax.

There are code security vulnerabilities in Solidity.

This is also a major weakness of its competitiveness. There are many security vulnerabilities in Solidity due to improper usage, so developers need to be well-prepared for security testing before writing contracts in Solidity.

Solidity still has challenges in terms of performance.

The performance of Solidity compared to relatively simple programming languages like Python still needs improvement. With the growth of Ethereum, there is also a continuous increase in Ethereum-based applications. The performance issues of Solidity will affect the overall operational efficiency of Ethereum, thereby increasing network load and reducing the overall security of the Ethereum network.

From the above perspective, Solidity, as the most popular smart contract development language on Ethereum, is powerful enough but not perfect. Move language emerged in this context.

III Mango Programming Model

3.1 Resource-oriented programming

Mango Move has been specifically enhanced for financial, social, and gaming scenarios, introducing resource-oriented programming. For common scenarios such as FT and NFT, Mango Move defines data as resources, ensuring data security at the virtual machine level.

3.2 Pure static language

Dynamic dispatch is the cornerstone of Solidity, and all cross-contract invocations are implemented through dynamic dispatch. It is also the entry point for most security vulnerabilities, such as TheDAO attack, PolyNetwork cross-chain attack, and so on. Given the real experience of Solidity, Mango Move adopts a completely pure static implementation to better ensure the security of on-chain assets.

3.3 Formal verification

Formal verification, also known as FV (formal verification), refers to the use of mathematical tools to analyze the space of possible behaviors of a design, rather than computing specific values as results. Mango Move comes with a formal verification tool that can be used to test and prove the reliability of contracts using mathematical methods.

3.4 Distributed Storage

In the era of Web3.0, users have ownership of data. Solidity stores contract data in a centralized manner through Maps. When a contract has a vulnerability and the contract Owner's permissions are obtained, all user data will be compromised. Mango Move cleverly uses Resource to store data scattered under each user's Account, ensuring both data security and true ownership of data by all network nodes.

3.5 Generic programming

For security reasons, Mango Move has been designed as a purely static language, but this has not reduced its flexibility. Through generic programming, Mango Move ensures the scalability of contracts and increases code reusability.

Blockchain is a very important system, with a large amount of assets stored on the blockchain. Moreover, once these transactions on the blockchain are executed, they cannot be revoked or tampered with. It is necessary to ensure the normal operation of functions such as transaction storage on the chain. Mango Move is a smart contract language that can be compiled and run in a blockchain environment that has implemented Move VM. Mango Move and the MoveProver tool have emerged as a result.

IV Mango Move core design concept

Ensuring the security of smart contracts from the ground up, according to SlowMist's previous report, blockchain security incidents in 2021 resulted in losses exceeding 9.8 billion US dollars. As an emerging programming language, the Move language has also made breakthroughs and innovations in terms of security at different levels.

Just as what First-classResources really means as "digital assets are first-class citizens", Move is a smart contract language designed for manipulating digital assets. Compared to other languages, Move has a more native and low-level handling of token assets.

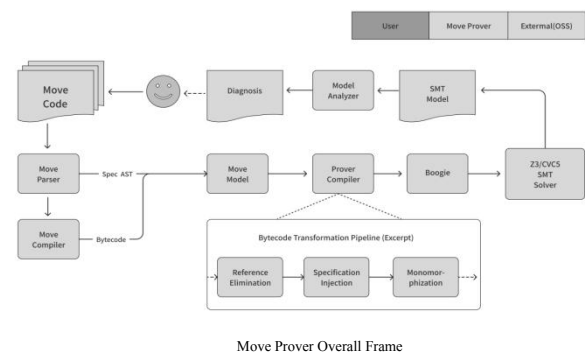
The Move language specifically defines assets as a type of resource, distinct from other data. In the context of blockchain, tokens are a type of resource, and the data of the resource must be stored under an account. During the transaction process, assets must flow to a destination, either transferred to another address or destroyed. Tokens cannot be duplicated or "double used".

Solidity and most programming languages treat tokens as numerical variables, where assets are simply numbers that can be added or subtracted. When the balance of one address decreases, the balance of another address increases. By using code, the decrease and increase in numbers can be made consistent, thus ensuring the security of assets through logical programming. Resource

encapsulates the concept of assets at the language level, avoiding the arbitrary generation and access of assets, greatly improving security.

Byte verification goes before contract execution. Move differs from Solidity in terms of compiler. As an executable bytecode language, Move has built-in security algorithms and bytecode verifiers, which can prevent many common errors.

The Move contract code must be verified before it can be executed, which allows the contract to be protected from potential compiler errors and possible attacks. Move has been committed to establishing a standardized culture from the very beginning, with a standard library for each Move module. The Move development team has initially developed a dedicated validator called Move Prover for contract verification.

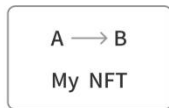


Based on modular composability: The modularity of smart contracts, which is more efficient and flexible, is another important feature for building a programming language ecosystem. The combination of smart contracts in the Solidity and other language ecosystems is primarily based on message passing between interfaces.

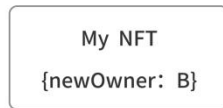
And in Mango Move, it is based on the combination of modules, interacting through the transfer of resources. Through the combination of modules, Mango Move saves block space occupied by contracts on one hand, and makes upgrades easier on the other hand. At the same time, due to the adoption of linear logic in the Module system, it can effectively package and encapsulate the concept of digital assets, clearly separating the definition of resources from the behaviors related to resources.

This resource-oriented programming brings about expressiveness and scalability that other languages cannot provide.

Transaction-Based



Resource-Based



Mango Move Partial Program Code Display

Current status of the Move Prover

- 100% open source and actively maintained on GitHub.
- Fully automatic verification that implementations meets specifications.
- Runs only a little slower than a linter (or other static analysis tools) — important for developer experience.
 - 8,800 lines of code.
 - 6,500 lines of specifications.
 - Verification completes in a few minutes.
- Enforced in continuous integration, every change of Move code needs to pass the verification.

Look and feel : programming in Move

```

module 0x1 : Account {
  // resources that can be stored in global storage
  struct Account has key {
    balance : u64
  }

  fun withdraw(account : address, amount : u64) acquires Account {
    // creates a mutable reference to an item in the global storage
    let ptr : &mut Account =
      borrow _ global_mut < Account > (account);
    ptr.balance = ptr.balance - amount;
  }

  // the "main" function for a transaction
  public entry fun transfer(
    from : address, into : address, amount : u64
  ) acquires Account {
    withdraw(from, amount);
    deposit(into, amount);
  }
}
  
```

V Type System of Mango Move

Due to a large amount of assets stored on the blockchain, the programming language of blockchain smart contracts must provide security guarantees, and Mango Move is a programming language designed for the blockchain that can provide better security.

The type system of Mango Move provides many protections, and we emphasize three points: first, type safety; second, resource safety; and finally, reference safety. The formal verification tool Move Prover mainly provides more advanced and expressive methods of representation. Whereas - structinvariants is a type, a structure should have some kind of state. -unit specification (per single function) Each program and each function must adhere to the corresponding specifications.

Type safety

Summary : Move is a strongly/statically typed language with strictly no type conversions

In more details :

- Every variable and expression in Move has one and only one type
- The type is known at compile time.
- The type can never be changed whatsoever.

NOTE: these are not type conversions as new variables are created

- freeze: &mut T --> & T
- as: u8/u64/u128 --> u8/u64/u128

5.1 Type safety

Mango Move is a strong type system and strictly guarantees that no type conversion can be done in any situation. Types cannot be converted between each other, and there is only one type for each. This is a guarantee provided by the Mango Move type system.

5.2 Resource safety

One is to ensure the scarcity of resources. The

abilities of copy and drop are depended to ensure that a resource is not duplicated, which is determined by these two abilities. The other two abilities (key and store) determine the possible use cases for this resource, whether it can appear in a function and disappear after the smart contract ends, and whether it can be written on the blockchain. These two abilities also determine this.

Mango Move also encapsulates resources, and all code related to resources should be encapsulated in one module, ensuring that developers cannot package it into multiple modules for resource handling. Therefore, Mango Move actually guarantees the security of resources in this aspect.

5.3 Quote safety

In the Mango Move language, it is ensured through "quote safety" and follows the ownership rules of Rust. Any local variable in a function can be modified by only one process at any given time. At the same time, for a global variable, within the entire blockchain, there can only be one owner who can make changes for any given type. This is ensured through Mango Move's reference safety.

Resource safety

Summary: Move ensures that there are no dangling references to both function locals and global storage — via ownership rules.

In more details :

- Any function local variable is uniquely owned at any time.
- Any Global $\langle T \rangle$ is uniquely owned at any time.

By uniquely owned, it means that a slot has:

- at most one writer and no readers, OR
- $N(> 0)$ readers and no writers.

VI Mango formalized verification system.

As a new generation of smart contract

programming language, Move has made security its top design goal in order to support Libra's vision as a financial infrastructure and empower billions of people worldwide. The security features of the Move language can be divided into three levels: language level, virtual machine level, and tool level.

The formalization verification tool for Move is called Move Prover. The basic idea is to use an automated theorem prover in the formal verification domain to verify whether a program complies with a certain specification. This method requires users to have a detailed understanding of the program's operational logic, representing the program logic as specifications and conveying it to the verification system along with the program. "Move defines a specification language called Move specification language." Translate the following text into en-US without explanation: Convert Move programs and specifications into Boogie programs using the Move to Boogie compiler - an intermediate verification language with formal semantics.

Outline

- 1 Introduction
- 2 Move type system
- 3 Move formal verification system

6.1 Struct invariant

The invariants of a struct allow for a complex struct with multiple fields, where there exists a relationship between different fields within the struct. The invariants of the struct allow for specifying the relationship between different fields, which is currently not achievable by the type system.

Outline

Struct invariants allows you to specify complicated relations among the fields of a struct type which have to hold at runtime.

This set of specifications for struct plus struct actually serves as an "enhanced type" that can ensure the relationships between fields within the struct conform to a certain relationship.

```

module 0x1 :: Account {
  struct SumsConst {
    a:u64,
    b:u64,
  }
  spec SumsConst {
    invariant a >= b;
    invariant a + b == 100;
  }

  fun create_valid(x : u64) : SumsConst {
    assert!(x >= 50, 1);
    SumsConst { a : x, b : 100 - x }
  }
}

```

```
[INFO] transforming bytecode
[INFO] generating verification conditions
[INFO] 1 verification conditions
[INFO] running solver
[INFO] 0.011s build,0.005s trafo,0.052s gen, 0.625s verify, total 0.693s
error: data invariant does not hold
    test _sum_ is _const. move : 7 : 9
        invariant a >= b :

at test _sum_ is _const. move : 11 : create _valid
    x = 47

at test _sum_ is _const. move : 12 : create _valid
at test _sum_ is _const. move : 13 : create _valid
at test _sum_ is _const. move : 7

exiting with verification errors
make: *** [ Makefile: 26 : demo _sum_ is _const] Error 1
```

6.2 Function specification

One focus of Mango Move is to do the specification of functions, specifying how a function should behave in certain situations, including how it should present itself and when it should throw an error. Once a warning condition is

triggered, the execution will be terminated, which is the goal of function specification.

Function specification

For people not familiar with formal verification, function specification can be loosely considered as exhaustive unit testing.

The unit testing joke

A software engineer walks up to an ATM and ...

```
- withdraw(@0x1, 10);
- withdraw(@0x1, 1000);
- withdraw(@0x1, 9999999999999999999999999999999);
- withdraw(@0x1, 0);
- withdraw(@0x1, 20);
- withdraw(@0xdeadbeef, -123);
- withdraw(@0xbaadf00d, 666 * 2);
```

6.2.1 Abort Conditions

Move Prover provides an opportunity to express function transactions in a more specialized manner, allowing developers to write specifications for functions through the function's specification.

Abort conditions in the function specification

```

module 0x1 : Account {
  struct Account has key { balance : u64 }

  fun withdraw(account : address, amount : u64) acquires Account {
    let ptr : &mut Account = borrow _ global _ mut < Account > (account);
    ptr.balance = ptr.balance - amount;
  }

  spec withdraw {
    aborts _ if !exists < Account > (account);
    aborts _ if global < Account > (account).balance < amount;
  }
}

```

6.2.2 Post-conditions

Post-condition is the running state of a program function.

Post conditions in the function specification

```
module 0x1 : Account {
  struct Account has key { balance : u64 }

  fun withdraw(account : address, amount: u64) acquires Account {
    let ptr : &mut Account = borrow _ global _ mut < Account > (account);
    ptr.balance = ptr.balance - amount;
  }
  spec withdraw {
    aborts _ if !exists < Account > (account);
    aborts _ if global < Account > (account).balance < amount;

    // on successful function return
    ensures global < Account > (account).balance ==
      old(global < Account > (account).balance) - amount;
  }
}
```

6.2.3 Pre-conditions

Function specification: Precondition, which means that in addition to being able to write exceptions in certain situations, there will be changes in the absence of exceptions. When calling this function, the precondition must exist. Having a precondition can simplify exception conditions.

6.2.4 Global Invariants

Invariants are a type of property that always remains the same during program execution. Immutability is one of them: once an object of an immutable type is created, it represents an unchanging value.

```
script {
  use 0x1::Debug;

  const RECEIVER : address = 0x999;

  fun main(account: &signer) {
    Debug::print<address>(&RECEIVER);

    // they can also be assigned to a variable

    let _ = RECEIVER;

    // but this code leads to compile error
    // RECEIVER = 0x800;
  }
}
```

VII Mango modular blockchain

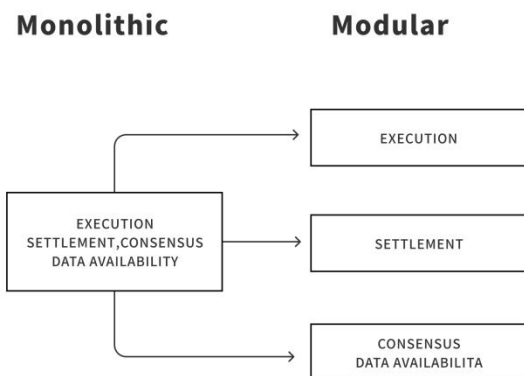
The Mango modular blockchain underlying code and contract layer code are programmed using the Move language, decomposing blockchain functionality into different levels of network architecture. Traditional blockchain systems typically integrate consensus, settlement, data availability, and execution functionalities into a single architecture. However, with the increasing complexity and demands of blockchain applications, a single architecture may not be able to meet the requirements of different scenarios. Therefore, Mango modular blockchain separates these core functions, allowing each functional module to operate independently while maintaining collaboration between them. This architecture makes the blockchain system more flexible and scalable, allowing for customization and optimization based on different needs.

7.1 Modular blockchain and monolithic blockchain

The concept of Mango modular blockchain corresponds to a single-piece blockchain, and the two have significant differences in terms of functionality processing. First, it is necessary to clarify the four core functions of blockchain:

1. Execution - transaction processing and computation.
2. Resolve - Dispute resolution and bridging.
3. Consensus - Transaction ordering.
4. Data availability - Ensure data is available.

Mango modular blockchain is any chain that is part of the modular blockchain stack, by modularizing the blockchain and splitting the four functionalities into multiple specialized layers. The modular blockchain stack consists of multiple layers of modular blockchains that depend on each other to create a system with all the above components.



7.2 Advantages of Mango Modular Blockchain

7.2.1 Sovereign protection

Although other layers are used, the Mango modular blockchain can have sovereignty like Layer1. This allows the blockchain to respond to hacker attacks and push upgrades without any underlying permission. Essentially, sovereign blockchain retains the ability to make critical decisions based on social consensus, which is one of the most important aspects of blockchain as a social coordination mechanism.

7.2.2 Efficient and low-cost start-up of new blockchains

Because the Mango modular blockchain does not need to handle all functions, new blockchains can simply use existing modular blockchains for components they wish to unload. This allows for the efficient bootstrapping of new blockchains, reducing deployment time and minimizing costs.

7.2.3 Scalability

Returning to the most fundamental issue,

solving scalability is a major reason for promoting the modular development of blockchain, without sacrificing security and decentralization. Modular blockchain is not limited by the requirement to handle all functionalities. By dividing them into multiple layers, scalability can be achieved without sacrificing security and decentralization. This makes sustainable blockchain scalable and compatible with a decentralized multi-chain environment.

VIII Mango's new paradigm for omni-chain applications.

8.1 From single chain to multi-chain, and then to omni-chain.

As the first smart contract platform, most of all decentralized applications could only be deployed on Ether at the beginning of its birth. However, with the flourishing of public chain ecosystems and the development of Layer 2 solutions, applications now have more choices.

8.1.1 Single chain deployment status

The application can choose to deploy on a specific chain that suits its own business. Each chain or L2 has its own unique mechanism design, with different characteristics in terms of decentralization, privacy, and data availability, etc., to meet the needs of different applications. At the same time, various one-click chain creation stacks are becoming more and more mature, and application developers can completely create their own chains and customize related features. A typical example is the DYDX protocol, which is an early decentralized derivatives trading market created on Ethereum and has now migrated to a separate Cosmos Zone to better meet its throughput requirements.

8.1.2 Multi-chain deployment status

Applications can choose to deploy on multiple chains, allowing users on different chains to access the services provided by the application. From the point of view of the application itself, it is possible to scale up the business and increase the revenue of the protocol. For example, AAVE, a well-known lending protocol, has deployed its application on six chains.

8.2 Multi-chain application VS Mango omni-chain application

The deployment of contracts on multiple chains in an application cannot share liquidity, which also leads to the fragmentation of the Web3 ecosystem, including the fragmentation of user experience and liquidity.

8.2.1 Fragmented user experience in multi-chain cross-chain

In order to use different on-chain services, users need to transfer assets across multiple chains, register addresses on different chains, and learn operations on different chains. In many cases, users need to take many steps and prepare many kinds of Gas in order to achieve the desired results.

8.2.2 Liquidity fragmentation across multiple chains

In many types of DeFi protocols, liquidity depth is a part of their core experience. The same liquidity cannot exist simultaneously on multiple chains. Every DeFi protocol, when deployed on a new chain, needs to rebuild liquidity, which reduces the overall efficiency of liquidity.

8.2.3 Mango Omni-dApp (Mango omni-chain application)

Mango's omni-chain application is based on

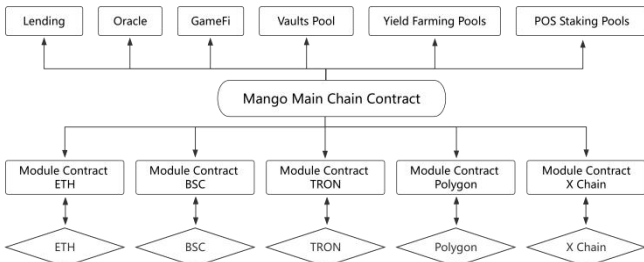
the Mango Move underlying modular deployment contract protocol to carry the program's global state, which is a new way of building applications where developers design the application as a whole, taking omni-chain interoperability as a prerequisite, rather than simply replicating a single-chain application on multiple chains. The Mango omni-chain application consists of different modules on different chains, which have interoperability and together form a complete application. Compared to multi-chain deployment, Mango's omni-chain application can extend its tentacles to more chains, allowing users on more chains to use program services without the problem of fragmented user experience and fragmented liquidity.

Features	Multi-Chain	Omni-Chain
Status record	Distributed across multiple chains.	Unified on one chain
Liquidity	Distributed across multiple chains.	Unified on one chain
User experience	Users need to be familiar with multi-chain operations, prepare multi-chain gas, and frequently transfer assets.	Users can access programs on any chain just like accessing local programs.
Integratability	Only friendly to integrated chains, but more complex for cross-chain integration.	Better cross-chain interoperability

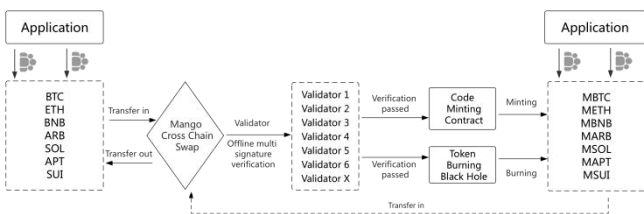
In this table, the analogy between omni-chain architecture and simple multi-chain deployment is shown:

8.3 Technical architecture of Mango Omni-chain Application

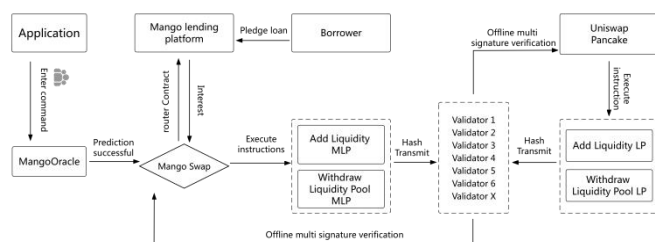
The technical architecture of the omni-chain application consists of the Mango main chain contract and module contract pattern. The main logic of the application is placed on the Mango main chain, like a "master control", and then other chains provide a remote access module to achieve interaction with end users, obtain user input, and output the desired results for users.



Mango Omni-chain Architecture Principle Logic Diagram



Mango's omni-chain cross-chain bridge technology architecture diagram



Mango's Omni-chain application technology architecture diagram

After the new chain obtains the user's input, it passes that input across the chain to the Mango master chain for processing and outputting the

result, which is then passed across the chain to the new chain for outputting to the user. In some cases, different modules of the main chain may be split into different chains, which together form a virtual main chain.

8.4 Technical architecture advantages of Mango's Omni-chain Application

1. The entire chain is easy to expand. The main logic of the program is processed on the Mango main chain, and the application has a unified state record. After deploying the contract on the new chain, users can inherit all the state records and liquidity from the main chain without reinventing the wheel.
2. The overall user experience is improved, as users do not need to worry about which chain the program is deployed on. They can access the program from the main chain of the entire network, just like accessing a local program. There is no need to transfer assets back and forth, and there is no need to learn operations on multiple blockchains or prepare various types of gas.
3. The entire chain is easy to integrate across chains. When other applications integrate this program, they only need to connect on the main chain of the entire network to access all its features and liquidity, instead of having to connect separately to all other chains.

Mango Network's omni-chain application, as a new paradigm, has provided many new possibilities for us, supporting for arbitrary chain forging, redemption, and exchange. The application allows users to complete operations without perceiving any cross-chain processes.

When the omni-chain application becomes the mainstream paradigm of decentralized applications,

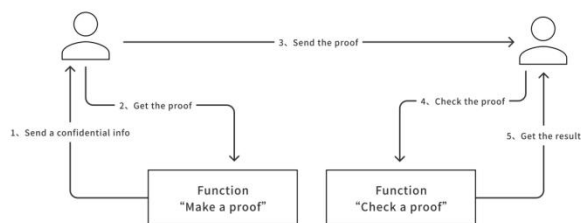
both application developers and users can experience unprecedented freedom in the blockchain world.

IX Mango public chain infrastructure protocol

9.1 ZK zero-knowledge proof

The Mango public chain combines zero-knowledge proofs, allowing one party to disclose information knowledge to another party without revealing the information itself. Extend the universal blockchain. Through proof of validity, blockchain nodes can verify transactions without storing detailed information or replaying computations. This reduces confirmation time and improves network throughput.

Due to its scalability advantages, zero-knowledge proof has become the core infrastructure of blockchain scaling projects, especially zero-knowledge rollups, in the Mango public chain. ZK-SNARKs and ZK-STARKs are the main types of zero-knowledge proofs.



9.1.1 Anonymous trading

Mango public chain uses ZK proof technology, allowing users to maintain anonymity during transactions. To verify the validity and legality of a

transaction without exposing any information about the transaction participants or the assets involved, generate ZK proofs.

9.1.2 Privacy protection

Mango's public chain uses ZK proof technology to perform data validation without revealing sensitive information. When verifying that a certain node on the chain owns specific assets, ZK proofs can be used without revealing the identity of the participant or any other sensitive information.

9.1.3 Data integrity

The Mango public chain ensures that important data such as block data and smart contract states have not been tampered with and have been correctly calculated through the zk-SNARKs (Zero-Knowledge Scalable Non-Interactive Argument of Knowledge) protocol, providing highly reliable data proof for the Mango public chain.

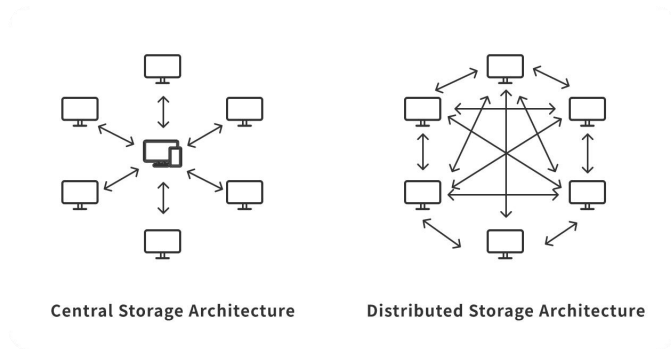
9.1.4 Cross-chain interaction

Mango public chain and ZK proof technology can be used for secure and private cross-chain interactions between different blockchains. To verify the validity of specific cross-chain transactions and protect the privacy of participants, it is done by generating ZK proofs.

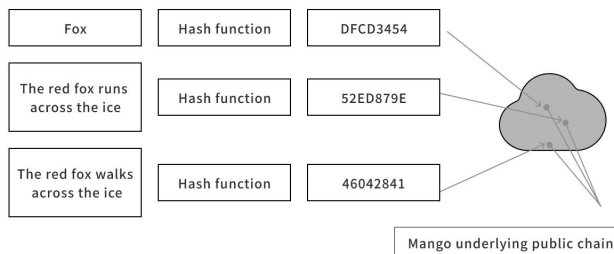
9.2 Distributed Storage

One way is to store data in a decentralized manner on multiple independent devices, which overall achieves distributed architecture. In the field of blockchain applications, it represents a new generation of distributed storage technology

represented by IPFS. Unlike traditional storage technology, the new generation of distributed storage not only changes the way storage is done, but also changes the system architecture and network transmission protocol, allowing distributed storage to truly be stored among different parties, while also achieving privacy protection and security for the data.



When the user extracts data, the same algorithm is used to calculate the hash of the data and obtain the corresponding data storage location from the hash table (as shown in the figure below).



In the Mango public chain infrastructure protocol, distributed storage is a technology used for storing and managing data. It disperses data storage across multiple nodes in the network to enhance data reliability, security, and scalability.

9.2.1 Data redundancy

The Mango public chain adopts distributed storage technology to achieve data redundancy. Each data block will be replicated on multiple different nodes to ensure that even if some nodes

fail or go offline, the complete data can still be obtained from other nodes.

9.2.2 Data encryption

In order to protect user privacy and data security, the Mango public chain infrastructure protocol uses encryption algorithms to encrypt the data uploaded by users to the distributed storage network, which ensures that only authorized users can access and decrypt this data.

9.2.3 Data reliability

By replicating data to multiple nodes and using fault-tolerant mechanisms to detect and repair damaged or lost copies, Mango public chain ensures that its distributed storage system has high reliability. Even if certain nodes fail or are attacked, it is still possible to recover lost or damaged files from other replicas.

9.2.4 Scalability

The distributed storage system of Mango public chain can be horizontally scaled according to demand. When more data storage is needed, simply adding more storage nodes can increase the storage capacity of the entire system without compromising performance or reliability.

9.2.5 Data permission control

The Mango public chain infrastructure protocol provides a powerful data permission control mechanism. Users can define the permissions and policies for accessing their data, ensuring that only authorized users can access and modify this data.

9.3 MgoDNS domain name service

MgoDNS is a solution for distributed domain names based on cross-chain protocols. It provides domain and domain data analysis services for non-intermediated networks. This platform can help businesses and individual users manage valuable on-chain data information, and now they can participate in digital asset transactions in a more efficient, secure, and convenient way. MgoDNS aims to make the internet a transparent, secure, and free space through its innovative decentralized measures.

MgoDNS maximizes compatibility and achieves decentralization of the system. The underlying layer of MgoDNS is blockchain, and the upper layer is traditional internet. MgoDNS can also connect various types of public chains and consortium chains, collectively forming a super hub for connecting various blockchains.

Traditional Internet has mature business models in project operation, but its drawbacks are also obvious, such as data leakage, server downtime, and other incidents. On the other hand, although blockchain technology fundamentally solves the problem of data trust, there is a lack of a certain foundation in terms of the scale and operation of applicable scenarios. So, the connection function of MgoDNS will enable the effective integration and application of both, complementing each other and achieving decentralization of the system.

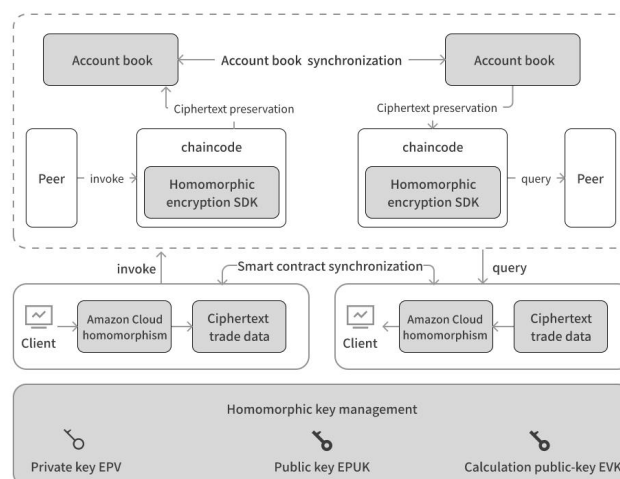
9.3.1 MgoDNS domain name system, connecting the information internet and the value internet.

Unlike the infrastructure of traditional domain name services, MgoDNS's smart contract can replace the roles and processes of most registration agencies. Any institution or individual can create subdomains based on the rules of the registering institution. The parser plays the role of a translator,

converting names into hash addresses and some mainstream public chain addresses. Based on cross-chain design, MgoDNS can serve existing public chain ecosystems such as Ethereum, IPFS, etc., making file access, address transfers, and smart contract calls more convenient and faster.

9.3.2 MgoDNS domain name parsing system, super hub connecting the information Internet and the value blockchain.

MgoDNS domain name system focuses on providing blockchain domain names and domain name data resolution services for blockchain networks, helping enterprises and individual users to participate in digital asset transactions and manage on-chain value data information more conveniently and securely. Meanwhile, based on cross-chain design, MgoDNS can serve existing public chain ecosystems such as Ethereum, IPFS, etc., making file access, address transfers, and smart contract calls more convenient and faster.



MgoDNS domain name resolution technology:

- MgoDNS achieves distributed domain name resolution by using domain name resolution smart contracts and cross-chain data exchange on multiple

mainstream chains.

- Domain name registration, resolution, and trading are not affected by any centralized factors, achieving complete decentralization. The security of domain names is guaranteed by blockchain technology, and the resolution of domain names is supported by a high-speed blockchain network.

- MgoDNS is fully compatible with the management and resolution of existing domain names.

- Support dynamic domain name resolution. MgoDNS automatically updates IP mapping and achieves blockchain confirmation in seconds through the wallet, enabling dynamic domain name services like traditional Peanut Shell and Dandelion without additional costs.

- Support for user-defined domain names. From the perspective of the domain name product, there are currently Ethereum Name Service (ENS) and Unstoppable Domains. However, ENS only supports the resolution of cryptocurrency addresses, making it difficult to connect to the traditional information internet and the underlying blockchain. On the other hand, the application scenarios of Unstoppable Domains are also relatively limited.

In addition to fulfilling the functions of traditional DNS domains, MgoDNS also combines the functions of information networks and value networks, supporting traditional domain name resolution, user identity authentication, DApp, social relationship networks, video conferences, and distributed information flow and other scenario-based applications.

	ENS	Unstoppable Domains	MgoDNS
Digital currency address resolution	Support	Support	Support
IP address parsing	Nonsupport	Support	Support
Traditional domain name analysis	Nonsupport	Nonsupport	Support
Underlying blockchain	Ethereum (ETH)	Ethereum (ETH)	ManGO
User identity authentication	Nonsupport	Nonsupport	Support
Domain name resolution client	Nonsupport	Nonsupport	Support
Decentralized network	Nonsupport	Support	Support
Decentralized encrypted mailbox	Nonsupport	Nonsupport	Support
Decentralized edge-cloud computing	Nonsupport	Nonsupport	Support
Decentralized edge DAppstore	Nonsupport	Nonsupport	Support

9.4 Mango client end

A copy (also known as: full client) is a consistent copy that maintains the effective state of the system, used for auditing and building transactions or operational services. Lightweight clients are based on object references and transactional authentication information, which can authenticate transactions that cause their creation or execution. Copies perform high-integrity reads of the system state without the need to maintain complete replica nodes. By providing concise

evidence packages and effect certificates, the lightweight client can undergo transformation within Mango and perform transaction operations or observe results. Regular checkpoint mechanism can be used to create collective checkpoints for final transactions, effectively verifying the recent state of objects and the events issued.

Customer-driven is a mechanism for handling authorization failures or client malfunctions. The client can solve these problems by updating to a level of honest authority that can handle correct transactions. Once there are no more certificates to synchronize, the list will be submitted to the new authority for execution. The relay is the client that performs this operation, and multiple relays can run simultaneously to update and coordinate the replica operation service with each other. Blocks are tools used by followers to receive updates and maintain the latest view state when processing authorizations. In addition, the authorities can use a push-pull gossip network to update transactions with each other and reduce the number of times the relay needs to perform this function. The authorities may use periodic status commitments to ensure that they have dealt with the complete set of certificates until a certain checkpoint.

The Mango system expands its capabilities by allocating more resources to handle transaction permissions. These resources include CPU, memory, network, and storage within or across multiple machines. Increasing resources can improve the ability to process transactions, thereby increasing revenue. At the same time, more resources will also reduce latency because operations can be executed without waiting for necessary resources to become available.

In order to ensure that more resources can increase capacity in a quasi-linear manner, the Mango system actively reduces bottlenecks and synchronizes points that require global locking within permissions. Transaction processing is explicitly divided into two phases: the first phase is to ensure exclusive access to a specific version of an object or shared access to an object; the second

phase is to execute the transaction and commit its effects.

For transactions involving shared objects, it is necessary to use consensus protocols for ordering, which may become a bottleneck. "But the latest high-throughput consensus protocol for engineering indicates that sorting is only the bottleneck in state machine replication, not the execution order." In the Mango system, sorting is only used to determine the version of the input shared object, that is, incrementing the object version number and associating it with the transaction summary, without involving sequential execution.

The second phase occurs when the versions of all input objects are known to the permissions and involves executing a move transaction and committing its effects. Once the version of the input object is known, it can be executed in a fully parallel manner. On multi-core or multi-physical machines, virtual machines read versioned input objects and write the generated objects to storage. For object and transaction storage, the consistency requirements are very loose (except for sequential lock mapping), allowing each authority to use scalable distributed key-value storage. In addition, due to the idempotent nature of the execution, it is easy to recover even in the event of component crashes or hardware failures.

X References

1. <https://github.com/MangoNet-Labs/mango>
2. D. Matsuoka, C. Dixon, E. Lazzarin, and R. Hackett.(2022) Introducing the 2022 state of crypto report. <https://a16z.com/tag/state-of-crypto-2022/>
3. S. Blackshear, E. Cheng, D. L. Dill, V. Gao, B. Maurer, T. Nowacki, A. Pott, S. Qadeer, D. R. Rain, S. Sezer, T. Zakian, and R. Zhou, "Move: A language with programmable resources," 2019. <https://developers.diem.com/papers/diem-move-a-language-with-programmableresources/2019-06-18.pdf>
4. Sam Blackshear, David L. Dill, Shaz Qadeer,

Clark W. Barrett, John C. Mitchell, Oded Padon, and Yoni Zohar. Submitted on July 23, 2020; v1 submitted on April 10, 2020; originally announced in April 2020. Resources: A Safe Language Abstraction for Money. CoRR abs/2004.05106 (2020). arXiv:2004.05106

<https://arxiv.org/abs/2004.05106>

5. Marco Patrignani and Sam Blackshear. 2021. Robust Safety for Move. CoRR abs/218.05043(2021).arXiv:218.05043

<https://arxiv.org/abs/218.05043>

6. Daniel Collins, Rachid Guerraoui, Jovan Komatovic, Petr Kuznetsov, Matteo Monti, Matej Pavlovic, Yvonne-Anne Pignolet, Dragos-Adrian Seredinschi, Andrei Tonkikh, and Athanasios Xygkis. 2020. Online Payments by Merely Broadcasting Messages. In the 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2020, Valencia, Spain, June 29 - July 2, 2020. IEEE

7. David Dill, Wolfgang Grieskamp, Junkil Park, Shaz Qadeer, Meng Xu, Emma Zhong. Fast and Reliable Formal Verification of Smart Contracts with the Move Prover. Submitted 12 February, 2022; v1 submitted 15 October, 2021; originally announced October.2021. arXiv:218.08362

<https://arxiv.org/abs/218.08362>